

Event-Type Player Similarity

Ziheng Lin

MLSE project

- **Question:** For a given player, who plays *most similarly* under a **specific event type** t (e.g. Pass, Carry)?
- **Contrast:** Global player similarity (one vector per player) mixes all behaviors; here we isolate **behavior-conditioned** similarity via profiles $v_{p,t}$.
- **Focus:** Methods, implementation, and empirical summaries from the built cache.

High-level pipeline

- 1 Load pretrained **EventEncoder** + `feature_vocab` from `models/event_encoder_mam.pt` (same checkpoint as training).
- 2 Stream processed events from JSONL (`events360_v4.jsonl`) — one JSON object per line.
- 3 For each event with `player.id` and `type.name`: encode $\rightarrow e \in \mathbb{R}^{128}$.
- 4 Online aggregate by key $(p, t) = (\text{player id}, \text{event type string})$: maintain **sum** and **count** only.
- 5 After one pass: profile $v_{p,t} = s_{p,t} / n_{p,t}$.
- 6 At query time: **cosine similarity** among $v_{.,t}$, with `min_count` on $n_{p,t}$.

EventEncoder: input $\rightarrow$ output (per event)

Input

- **Attributes:** flattened event fields $\rightarrow$ **integer IDs** (must match **training vocab**).
- **Space (360):** actor location; freeze_frame = visible players (positions + teammate / GK flags).

Output

- One vector $e_i \in \mathbb{R}^{128}$: merges “**what**” (attributes) and “**where**” (layout around the actor).
- Build step: **forward pass only**.

Method: online aggregation

For each (p, t) :

$$s_{p,t} = \sum_{i: \text{player}(i)=p, \text{type}(i)=t} e_i, \quad n_{p,t} = \#\{\text{such events}\}, \quad v_{p,t} = \frac{s_{p,t}}{n_{p,t}}.$$

- **Memory:** $O(\#\text{distinct } (p, t))$ — we never store all e_i .
- **Batches:** events are grouped into mini-batches (default 256); each batch forward pass updates many keys at once.

Method: query

Given query player q and event type t :

- Require $n_{q,t} \geq \text{min_count}$ for the query player (stability).
- For each candidate p : require a stored profile and $n_{p,t} \geq \text{min_count}$.
-

$$\text{sim}(p) = \cos(\mathbf{v}_{q,t}, \mathbf{v}_{p,t}) = \frac{\mathbf{v}_{q,t}^\top \mathbf{v}_{p,t}}{\|\mathbf{v}_{q,t}\|_2 \|\mathbf{v}_{p,t}\|_2}.$$

- Implementation: `l2_normalize` rows and query, then matrix–vector product (`cosine_scores`).
- Return top- k excluding self.

Why not PlayerBERT?

- **PlayerBERT** consumes *ordered* event sequences within a match; useful for temporal structure.
- Here we want a **marginal** “how does p look when doing t ?” $\Rightarrow$ mean of EventEncoder outputs over all t -events is a direct estimate.
- Keeps build **one pass**, cache **small** ($\sim 25\text{--}30$ MB for profiles vs. storing every event).

CLI: query one event type

```
.venv/bin/python facet_similarity_tools.py query \  
  --profiles models/player_eventtype_profiles.pt \  
  --player "Burak Yilmaz" \  
  --event-type "Pass" \  
  --top-k 10 --min-count 20
```

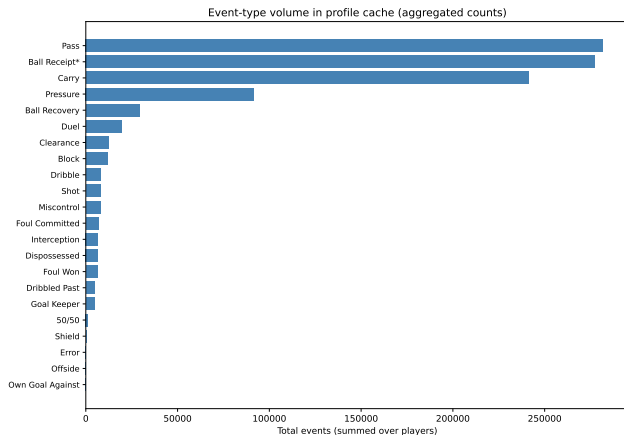
- query-all-types: repeat for every type t with $n_{q,t} \geq \text{min_count}$.
- query-global: nearest neighbors in `player_embeddings.pt` (original whole-player pipeline).

Empirical summary

Metric	Value
JSONL rows read	1,027,908
Events encoded	1,027,857
Skipped (no <code>player.id</code>)	51
Distinct players	2,409
Distinct <code>type.name</code>	22
Stored (p, t) profile cells	31,039

(Numbers from stats in `player_eventtype_profiles.pt`; regenerate figures after rebuild.)

Event-type volume

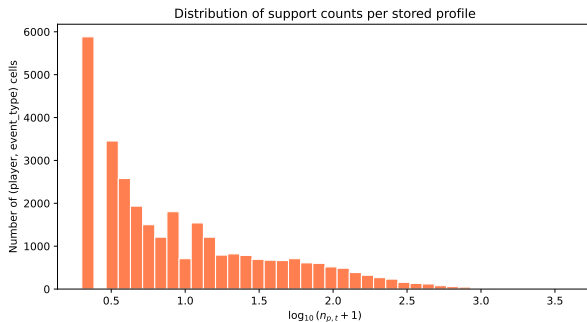


Top types by total events in cache:

- Pass: ~282k
- Ball Receipt*: ~278k
- Carry: ~241k
- Pressure: ~92k
- Ball Recovery: ~29k

Long tail: e.g. Offside, Own Goal Against are rare.

Distribution of support $n_{p,t}$



Histogram of $\log_{10}(n_{p,t} + 1)$ over all stored (p, t) cells.

- Many cells have moderate counts; tail reflects frequent types (e.g. Pass) for active players.
- Use `-min-count` at query time to ignore unstable estimates.

Worked example: query output

Example command: query for a forward, Pass, top-k=5, min-count=20.

```
Top 5 similar players ... for event_type=Pass (query_count=49):  
  Alexandra Morgan Carrasco (player_id=5085, sim=0.9850, count=75)  
  Borja Iglesias Quintas (player_id=11391, sim=0.9844, count=74)  
  Harry Kane (player_id=10955, sim=0.9913, count=21)  
  Aitana Bonmati Conca (player_id=15284, sim=0.9899, count=37)  
  Alphonso Davies (player_id=12365, sim=0.9899, count=22)
```

sim: cosine similarity in $[0, 1]$; count: $n_{p, \text{Pass}}$ for that neighbor.

Worked example: interpreting neighbors

- Under **Pass**, neighbors can span leagues and genders — the model encodes *event + 360 context*, not demographics.
- High sim with diverse names is expected if the embedding space clusters “how” the pass looks, not “who”.
- Always check count: prefer $\text{min-count} \geq 20\text{--}50$ for stable rankings.

Comparison: query vs. query-global

Mode	What it compares
query	$v_{q,t}$ vs. $v_{p,t}$ for fixed t (event-type conditioned).
query-global	Whole-player vector from <code>player_embeddings.pt</code> (mixed behaviors).

Use event-type queries for “who passes like X?”; use global for overall style similarity.

- **Mean pooling:** $v_{p,t}$ is one average over *all* t -events for player p (all matches, all situations).
- **Mixed sub-styles:** same `type.name` can hide different habits (e.g. circulation vs. line-breaking); they collapse into one marginal vector.
- **Sparse** (p, t): rare types stay noisy even near `min_count`; interpret neighbors carefully.
- **Checkpoint:** load `feature_vocab` + `EventEncoder` from the **same** `event_encoder_mam.pt`; mixing files breaks IDs and weights.

- **Aggregation:** trimmed / weighted means; optional bins by match phase or pitch zone.
- **Facets:** condition on extra fields (e.g. `pass.height.name`, body part), not only `type.name`.
- **Normalization:** per-league or league-invariant scaling if cross-league comparison matters.
- **Sequences:** PlayerBERT for temporal structure—complements marginal (p, t) profiles.

- **Encode:** frozen EventEncoder $\rightarrow e_i \in \mathbb{R}^{128}$ per event.
- **Aggregate:** online mean $v_{p,t}$ per (player p , type t).
- **Query:** cosine top- k on $v_{.,t}$ for fixed t ; min_count on query and candidates.
- **Build:** stream JSONL; memory scales with distinct (p, t) , not every event row.